NumPy

The NumPy (Numerical Python) library first appeared in 2006 and provides preferred Python array implementation as it offers a high-performance, richly functional n-dimensional array type called ndarray. Operations on ndarrays are up to two orders of magnitude faster than those on lists. In a big-data world in which applications may do massive amounts of processing on vast amounts of array-based data, this performance advantage can be critical.
A strength of NumPy is “array- oriented programming,” which uses functional-style programming with internal iteration to make array manipulations concise and straightforward, eliminating the kinds of bugs that can occur with the external iteration of explicitly programmed loops.
Many popular data science libraries such as Pandas, SciPy (Scientific Python) and Keras (for deep learning) are built on or depend on NumPy.

Creating arrays from existing data

The NumPy documentation recommends importing the numpy module as `np` so that you can access its members with `np.`

```
>>> import numpy as np
```

The numpy module provides various functions for creating arrays. Here we use the array function, which receives as an argument aan array or other collection of elements and returns a new array containing the argument’s elements.

```
>>> import numpy as np
>>> nums = np.array([2, 4, 6, 8, 10])
>>> type(nums)
<class 'numpy.ndarray'>
>>> nums
array([ 2,  4,  6,  8, 10])
```

- When outputting an array, NumPy separates each value from the next with a comma and a space and right-aligns all the values using the same field width. It determines the field width based on the value

that occupies the largest number of character positions. In this case, the value 11 occupies the two character positions, so all the values are formatted in two-character fields. That's why there's a leading space between the [and 2.

Multidimensional arguments

The array function copies its argument's dimensions.

```
>>> nums = np.array([[1, 2, 3], [4, 5, 6]])
>>> nums
array([[1, 2, 3],
       [4, 5, 6]])
```

- NumPy auto-formats arrays, based on their number of dimensions, aligning the columns within each row.

Exercises

Q: Create a one-dimensional array from a list comprehension that produces the even integers from 2 through 20.

```
>>> import numpy as np
>>> nums = np.array([i for i in range(2, 21, 2)])
>>> nums
array([ 2,  4,  6,  8, 10, 12, 14, 16, 18, 20])
```

Q: Create a 2-by-5 array containing the even integers from 2 through 10 in the first row and the odd integers from 1 through 9 in the second row.

```
>>> import numpy as np
>>> nums = np.array([[n for n in range(2, 11, 2)], [m for m
in range(1, 10, 2)]])
>>> nums
array([[ 2,  4,  6,  8, 10],
       [ 1,  3,  5,  7,  9]])
```

Array attributes

An array object provides attributes that enable you to discover information about its structure and contents.

`dtype`: determine array's type

```
>>> import numpy as np
>>> integers = np.array([[1, 2, 3], [4, 5, 6]])
>>> floats = np.array([0.0, 0.1, 0.2, 0.3, 0.4])
>>> integers.dtype
dtype('int64')
>>> floats.dtype
dtype('float64')
```

- `int64` -> It means every element in `integers` occupies 64 bits or 8 bytes in system memory.
- The size is platform dependent. Hence, `int32` in 32-bit operating systems.
  Most commonly you'll see the types `int64`, `float64`, `bool` (for Boolean) and `object` for non-numeric data (such as strings).

`ndim`, `shape`: determine array's dimensions

The attribute `ndim` contains an array's number of dimensions and the attribute `shape` contains a tuple specifying an array's dimensions

```
# using previously defined integers and floats arrays
>>> integers.shape
(2, 3)
>>> floats.ndim
1
>>> floats.shape
(5,)
>>>
```

- Here, integers has 2 rows and 3 columns (6 elements) and floats is one-dimensional.

`size`, `itemsize`: determine array's number of elements and element size

```
# using previously defined integers and floats arrays
>>> integers.size
6
>>> integers.itemsize
8
>>> floats.size
5
>>> floats.itemsize
8
```

Filling arrays with specific values

NumPy provides functions `zeros`, `ones` and `full` for creating arrays containing 0s, 1s or a specified value, respectively.
By default, zeros and ones create arrays containing `float64` values.

`zeros`

```
>>> import numpy as np
>>> zeros = np.zeros(10)
>>> zeros
array([0., 0., 0., 0., 0., 0., 0., 0., 0., 0.])
>>> >>> zeros = np.zeros(10, dtype=int)
>>> zeros
array([0, 0, 0, 0, 0, 0, 0, 0, 0, 0])
>>> zeros = np.zeros((2, 3), dtype=int)
>>> zeros
array([[0, 0, 0],
       [0, 0, 0]])
```

`ones`

```
>>> ones = np.ones((3, 3), dtype=int)
>>> ones
array([[1, 1, 1],
       [1, 1, 1],
       [1, 1, 1]])
```

`full`

The array returned by full contains elements with the second argument's value and type

```
>>> nums = np.full((3, 3), -1)
>>> nums
array([[-1, -1, -1],
       [-1, -1, -1],
       [-1, -1, -1]])
```

Creating arrays from ranges

NumPy provides optimised functions for creating arrays from ranges. NumPy also supports nonlinear ranges.

`arange()`

NumPy's `arange` function to create integer ranges—similar to using built-in function `range`.
In each case, `arange` first determines the resulting array's number of elements, allocates the memory, then stores the specified range of values in the array:

```
>>> import numpy as np
>>> np.arange(5)
array([0, 1, 2, 3, 4])
>>> np.arange(1)
array([0])
>>> np.arange(10, 0, -2)
array([10,  8,  6,  4,  2])
>>> np.arange(5, 10)
array([5, 6, 7, 8, 9])
```

- Though you can create arrays by passing `ranges` as arguments, always use `arange` as it's optimised for arrays.

`linspace`

You can produce evenly spaced floating-point ranges with NumPy's `linspace` function.

The function's first two arguments specify the starting and ending values in the range, and the ending value is included in the array. The optional keyword argument `num` specifies the number of evenly spaced values to produce—this argument's default value is 50.

```
>>> np.linspace(0.0, 1.0)
array([0.        , 0.02040816, 0.04081633, 0.06122449,
0.08163265,
       0.10204082, 0.12244898, 0.14285714, 0.16326531,
0.18367347,
       0.20408163, 0.2244898 , 0.24489796, 0.26530612,
0.28571429,
       0.30612245, 0.32653061, 0.34693878, 0.36734694,
0.3877551 ,
       0.40816327, 0.42857143, 0.44897959, 0.46938776,
0.48979592,
       0.51020408, 0.53061224, 0.55102041, 0.57142857,
0.59183673,
       0.6122449 , 0.63265306, 0.65306122, 0.67346939,
0.69387755,
       0.71428571, 0.73469388, 0.75510204, 0.7755102 ,
0.79591837,
       0.81632653, 0.83673469, 0.85714286, 0.87755102,
0.89795918,
       0.91836735, 0.93877551, 0.95918367, 0.97959184, 1.
])
>>> np.linspace(0.0, 1.0, num=5)
array([0.  , 0.25, 0.5 , 0.75, 1.  ])
```

Array Operators

NumPy provides many operators which enable you to write simple expressions that perform operations on entire arrays.

Arithmetic operations with arrays and individual numeric values

Element-wise arithmetic performs element-wise operations which are applied to every element in the array, each returns a new array containing the result.

```
>>> import numpy as np
>>> nums = np.arange(1, 10)
>>> nums
array([1, 2, 3, 4, 5, 6, 7, 8, 9])
>>> nums + 2
array([ 3,  4,  5,  6,  7,  8,  9, 10, 11])
>>> nums - 3
array([-2, -1,  0,  1,  2,  3,  4,  5,  6])
>>> nums * 2
array([ 2,  4,  6,  8, 10, 12, 14, 16, 18])
>>> nums ** 3
array([  1,   8,  27,  64, 125, 216, 343, 512, 729])
```

Augmented assignments modify every element in the left operand.

```
>>> nums
array([1, 2, 3, 4, 5, 6, 7, 8, 9])
>>> nums += 5
>>> nums
array([ 6,  7,  8,  9, 10, 11, 12, 13, 14])
```

Arithmetic operations between arrays

You may perform arithmetic operations and augmented assignments between arrays of the same shape.

```
>>> nums1 = np.arange(0, 5)
>>> nums2 = np.arange(5, 10)
>>> nums1.size
5
>>> nums2.size
5
>>> nums1 + nums2
array([ 5,  7,  9, 11, 13])
>>> nums2 - nums1
array([5, 5, 5, 5, 5])
>>> nums1 * nums2
array([ 0,  6, 14, 24, 36])
>>> nums1 / nums2
```

```
array([0.        , 0.16666667, 0.28571429, 0.375     ,
0.44444444])
>>> nums1 % nums2
array([0, 1, 2, 3, 4])
```

Arithmetic between arrays of integers and floating-point numbers results in an array of floating-point numbers.

Arithmetic operations of arrays with dissimilar shape

```
>>> nums1 = np.array([1, 2, 3, 4])
>>> nums2 = np.array([6, 8, 9])
>>> nums1 + nums2
Traceback (most recent call last):
  File "<python-input-80>", line 1, in <module>
    nums1 + nums2
    ~~~~~~^~~~~~~
ValueError: operands could not be broadcast together with
shapes (4,) (3,)
```

Broadcasting

Normally, the arithmetic operations require as operands two arrays of the same size and shape.
When one operand is a single value, called a ***scalar***, NumPy performs the element- wise calculations as if the scalar were an array of the same shape as the other operand. This is referred to as ***Broadcasting***.

```
>>> nums1 = np.array([1, 2, 3, 4])
>>> nums2 = np.array([6, 8, 9])
>>> nums1 + nums2
Traceback (most recent call last):
  File "<python-input-80>", line 1, in <module>
    nums1 + nums2
    ~~~~~~^~~~~~~
ValueError: operands could not be broadcast together with
shapes (4,) (3,)
>>> num2 = np.array([2])
```

```
>>> nums1 + num2
array([3, 4, 5, 6])
```

Comparing arrays

You can compare arrays with individual values and with other arrays. Comparisons are performed element-wise.
Such comparisons produce arrays of Boolean values in which each element's `True` or `False`.

```
>>> nums = np.arange(11, 21)
>>> nums
array([11, 12, 13, 14, 15, 16, 17, 18, 19, 20])
>>> nums ⩾ 14
array([False, False, False,  True,  True,  True,  True,
True,  True,
        True])
>>> nums % 2 == 0
array([False,  True, False,  True, False,  True, False,
True, False, True])
>>> nums
array([11, 12, 13, 14, 15, 16, 17, 18, 19, 20])
>>> nums2 = np.arange(20, 10, -1)
>>> nums2
array([20, 19, 18, 17, 16, 15, 14, 13, 12, 11])
>>> nums ⩾ nums2
array([False, False, False, False, False,  True,  True,
True,  True, True])
```

Exercises

Q: Create an array of the values from 1 through 5, then use broadcasting to square each value.

```
>>> np.arange(1, 6) ** 2
array([ 1,  4,  9, 16, 25])
```

NumPy Calculation Methods

We can use methods to calculate `sum`, `min`, `max`, `mean`, `std` (standard deviation) and `var` (variance)—each is a functional-style programming reduction:

```
>>> grades
array([56, 70, 81, 45])
>>> grades.sum()
np.int64(252)
>>> grades.min()
np.int64(45)
>>> grades.max()
np.int64(81)
>>> grades.mean()
np.float64(63.0)
>>> grades.std()
np.float64(13.656500283747663)
>>> grades.var()
np.float64(186.5)
```

Calculations by row or column

Many calculation methods can be performed on specific array dimensions, known as the ***array's axes***.
These methods receive an axis keyword argument that specifies which dimension to use in the calculation, giving you a quick way to perform calculations by row or column in a two-dimensional array.

- `axis=0` performs the calculation on all the row values within each column
- Similarly, specifying `axis=1` performs the calculation on all the column values within each individual row.

```
>>> grades1
array([56, 70, 81, 45])
>>> grades2
array([66, 72, 40, 31])
>>> grades3 = np.array([grades1, grades2])
>>> grades3
array([[56, 70, 81, 45],
       [66, 72, 40, 31]])
```

```
>>> grades3.mean()
np.float64(57.625)
>>> grades3.mean(axis=0)
array([61. , 71. , 60.5, 38. ])
>>> grades3.mean(axis=1)
array([63.  , 52.25])
```

- `grades3.mean()` -> `(grades1.mean()+grades2.mean()) / 2`
- `grades3.mean(axis=0)` -> `[(56 + 66) / 2, (70 + 72) / 2, ...]`
- `grades3.mean(axis=1)` -> `[grades1.mean(), grades2.mean()]`

Indexing and Slicing

One-dimensional arrays can be indexed and sliced using the same syntax and techniques as in with regular list or tuple types.

Indexing with Two-Dimensional arrays

To select an element in a two-dimensional array, specify a tuple containing the element's row and column indices in square brackets

```
>>> import numpy as np
>>> nums = np.array([[1, 3, 5, 7, 9], [2, 4, 6, 8, 10]])
>>> nums[0, 3]    # 0 → first row, 3 → fourth item
np.int64(7)
>>> nums[1, -1]    # 1 → second row, -1 → last item
np.int64(10)
```

Selecting subset of a two-dimensional array's rows

```
>>> import numpy as np
>>> nums = np.array([[1, 3, 5, 7, 9], [2, 4, 6, 8, 10],
[12, 15, 17, 19, 21]])
>>> nums
array([[ 1,  3,  5,  7,  9],
       [ 2,  4,  6,  8, 10],
       [12, 15, 17, 19, 21]])
>>> nums[1]
array([ 2,  4,  6,  8, 10])
```

```
>>> nums[1:3]
array([[ 2,  4,  6,  8, 10],
       [12, 15, 17, 19, 21]])
>>> nums[[0, 2]]
array([[ 1,  3,  5,  7,  9],
       [12, 15, 17, 19, 21]])
```

- `nums[1]` -> grab the second row
- `nums[1:3]` -> grab the second(index=1) to third row(index=2)
- `nums[[0, 2]]` -> grab the first(index=0) and third row(index=2), skipping the second

Selecting subset of a two-dimensional array's columns

You can select subsets of the columns by providing a tuple specifying the row(s) and column(s) to select. Each can be a specific index, a slice or a list.

```
>>> nums
array([[ 1,  3,  5,  7,  9],
       [ 2,  4,  6,  8, 10],
       [12, 15, 17, 19, 21]])
>>> nums[:, 0]    # select the first column from all rows
array([ 1,  2, 12])
>>> nums[[1, 2], 0]    # select the first column from
second and third row
array([ 2, 12])
>>> nums[:, 3:]    # select the last two columns from all
the rows
array([[ 7,  9],
       [ 8, 10],
       [19, 21]])
>>> nums[:2, [1, 3]]    # select the second and fourth
column from the first two rows
array([[3, 7],
       [4, 8]])
```

Copying

Views: Shallow Copies

View objects are objects that monitors the data in other objects rather than having their own copies of the data.
Views are also referred to as ***Shallow Copies.***
Various array methods and slicing operations produce views of an array's data.

The ndarray method `view` **returns a new array object with a view of the original array object's data**.

```
>>> import numpy as np
>>> nums = np.arange(1, 6)
>>> nums
array([1, 2, 3, 4, 5])
>>> nums2 = nums.view()
>>> nums2
array([1, 2, 3, 4, 5])
```

To verify that `nums` & `nums2` are indeed **different** ndarray objects we can use the built-in function `id`.

```
>>> id(nums)
139866041341136
>>> id(nums2)
139866041341424
```

Though views are separate array objects, they save memory by sharing element data from other arrays.

Now, to verify that `nums2` ndarray elements are not just copied data, modify `nums` and check.

```
>>> nums[1] *= 10
>>> nums
array([ 1, 20,  3,  4,  5])
>>> nums2
array([ 1, 20,  3,  4,  5])
```

- the second element is multiplied with 10 and the change is then reflected in both the ndarrays.

This process is vice-versa, it means that, changing a value in the view also changes the data in the original array.

```
>>> nums2[-1] /= 10
>>> nums2
array([ 1, 20,  3,  4,  0])
>>> nums
array([ 1, 20,  3,  4,  0])
```

Slicing also creates views, but only for the portion we have sliced.

```
>>> nums = np.arange(2, 10, 2)
>>> nums
array([2, 4, 6, 8])
>>> nums2 = nums[2:]
>>> nums2
array([6, 8])
>>> nums[-1] /= 4
>>> nums
array([2, 4, 6, 2])
>>> nums2
array([6, 2])
```

Deep Copy

The array method `copy` returns a new array object with a deep copy of the original array object's data.

```
>>> nums1 = np.arange(3, 30, 5)
>>> nums1
array([ 3,  8, 13, 18, 23, 28])
>>> nums2 = nums1.copy()
>>> nums2
array([ 3,  8, 13, 18, 23, 28])
>>> nums1[3] /= 9
>>> nums1
array([ 3,  8, 13,  2, 23, 28])
>>> nums2
array([ 3,  8, 13, 18, 23, 28])
```

- The fourth item remains unchanged in `nums2`

Reshaping and Transposing

Reshaping

`reshape`

Method `reshape` returns a view (shallow copy) of the original array with the new dimensions.
It does not modify the original array.

```
>>> grades = np.array([np.arange(2, 10, 2), np.arange(3,
11, 2)])
>>> grades
array([[2, 4, 6, 8],
       [3, 5, 7, 9]])
>>> grades.reshape(4, 2)
array([[2, 4],
       [6, 8],
       [3, 5],
       [7, 9]])
>>> grades.reshape(8, 1)
array([[2],
       [4],
       [6],
       [8],
       [3],
       [5],
       [7],
       [9]])
>>> grades.reshape(3, 5)
Traceback (most recent call last):
  File "<python-input-41>", line 1, in <module>
    grades.reshape(3, 5)
    ~~~~~~~~~~~~~~~^^^^^^
ValueError: cannot reshape array of size 8 into shape (3,5)
```

`resize`

`resize` works the same way as `reshape` with the only difference that `resize` modifies the original array's shape

```
>>> grades.resize(4, 2)
>>> grades
array([[2, 4],
       [6, 8],
       [3, 5],
       [7, 9]])
```

`flatten`

You can take a multidimensional array and flatten it into a single dimension with the method `flatten`.
It deep copies the original array's data

```
>>> grades
array([[2, 4, 6, 8],
       [3, 5, 7, 9]])
>>> grades.flatten()
array([2, 4, 6, 8, 3, 5, 7, 9])
```

`ravel`

Method `ravel` works the same as `flatten` with the only difference that it produces a view of the original array.

```
>>> grades
array([[2, 4, 6, 8],
       [3, 5, 7, 9]])
>>> raveled_grades = grades.ravel()
>>> raveled_grades
array([2, 4, 6, 8, 3, 5, 7, 9])
>>> grades[1][0] /= 3
>>> grades
array([[2, 4, 6, 8],
       [1, 5, 7, 9]])
>>> raveled_grades
array([2, 4, 6, 8, 1, 5, 7, 9])
```

Transposing

Transposing means "flipping" the array so the rows become the columns and the columns become the rows.
The `T` attribute returns a transposed view (shallow copy) of the array.
Transposing does not modify the original array.

```
>>> grades
array([[2, 4, 6, 8],
       [1, 5, 7, 9]])
>>> grades.T
array([[2, 1],
       [4, 5],
       [6, 7],
       [8, 9]])
```

Horizontal and Vertical Stacking

We can combine arrays by adding more columns or more rows—known as ***horizontal stacking*** and ***vertical stacking***.
`hstack` -> for horizontal stacking
`vstack` -> for vertical stacking

```
>>> grades
array([[1, 3, 5],
       [2, 4, 6]])
>>> grades2
array([[ 3,  5,  7],
       [ 6,  8, 10]])
>>> np.hstack((grades, grades2))
array([[ 1,  3,  5,  3,  5,  7],
       [ 2,  4,  6,  6,  8, 10]])
>>> np.vstack((grades, grades2))
array([[ 1,  3,  5],
       [ 2,  4,  6],
       [ 3,  5,  7],
       [ 6,  8, 10]])
```